TRAVAUX PRATIQUES

Bases de Données & Modélisation des SI

TP 8 : Trier et limiter les résultats avec ORDER BY LIMIT et OFFSET

Objectif : trier les produits sur une ou plusieurs colonnes, sélectionner les premiers résultats et parcourir une liste par pages dans AdventureWorks sous PostgreSQL.

Prérequis : SELECT, FROM, WHERE et filtres combinés. Le TP comprend 20 étapes guidées et cinq exercices à rendre.

I- Préparer le script et les colonnes

1. Ouvrez Query Tool sur AdventureWorks et enregistrez TP8_Nom_Prenom.sql. Repérez production.product et les colonnes productid, name, productsubcategoryid, listprice et color.

2. Affichez les colonnes utiles pour préparer vos contrôles.

```
SELECT productid, name, productsubcategoryid, listprice
FROM production.product;
```

Les noms sont écrits en minuscules. Adaptez les identifiants à votre sauvegarde si des guillemets doubles sont nécessaires. productsubcategoryid désigne une sous-catégorie. Les prix et effectifs observés dépendent de votre base : les valeurs annoncées dans la transcription ne constituent pas des résultats imposés.

II- Comprendre le tri et la place des clauses

ORDER BY ordonne les lignes du résultat. Il ne modifie pas le contenu ni le rangement permanent de la table. Sans cette clause, aucun ordre de restitution n'est garanti, même si plusieurs exécutions semblent afficher la même liste. Le mot-clé SQL à utiliser est ORDER BY, et non SORT BY.

```
SELECT productid, name, listprice
FROM production.product
WHERE listprice > 100
ORDER BY name ASC, productid ASC
LIMIT 10;
```

La requête conserve les prix supérieurs à 100, trie les lignes retenues, puis restitue au maximum dix lignes. WHERE vient avant ORDER BY ; LIMIT vient après le tri. Le point-virgule termine l'instruction entière. Une colonne de filtre peut être différente des colonnes de tri.

3. Exécutez cet exemple. Vérifiez le prix, le nombre maximal de lignes et la progression des noms.

LIMIT 10 ne signifie pas « les dix produits les moins chers » : ici, le premier critère de tri est le nom. Les dix lignes sont donc choisies au début de la liste alphabétique filtrée. L'ordre des textes dépend de la collation, notamment pour les accents et la casse.

III- Trier dans les deux sens

ASC signifie croissant et constitue le sens par défaut. DESC signifie décroissant. Pour des nombres, ASC va du plus petit au plus grand ; pour des dates, de la plus ancienne à la plus récente. Pour les noms, on parle usuellement de A à Z ou de Z à A, selon l'ordre de comparaison configuré.

4. Triez les noms en ordre croissant, puis en ordre décroissant.

```
SELECT productid, name
FROM production.product
ORDER BY name ASC, productid ASC
LIMIT 10;
```

```
SELECT productid, name
FROM production.product
ORDER BY name DESC, productid ASC
LIMIT 10;
```

5. Retirez ASC après name dans la première requête. Comparez : le sens de tri reste croissant.

Chaque colonne possède son propre sens. Dans ORDER BY name DESC, productid ASC, seul le nom est décroissant. La seconde colonne sert à départager des noms égaux. Préciser ASC reste utile pour rendre l'intention visible.

IV- Afficher les cinq produits les moins chers

6. Utilisez listprice comme premier critère de tri croissant.

```
SELECT productid, name, listprice
FROM production.product
ORDER BY listprice ASC, productid ASC
LIMIT 5;
```

Des produits à prix zéro peuvent apparaître. Ils satisfont le tri : le moteur ne devine pas que zéro doit être exclu. productid, identifiant unique, fixe un ordre entre des produits de même prix. Sans ce second critère, le choix parmi les ex æquo peut varier.

7. Si la consigne vise uniquement les prix strictement positifs, ajoutez le filtre adapté.

```
SELECT productid, name, listprice
FROM production.product
WHERE listprice > 0
ORDER BY listprice ASC, productid ASC
LIMIT 5;
```

Cette version exprime un autre périmètre : les produits à zéro et les prix NULL sont écartés par WHERE. Comparez les deux résultats et expliquez ce qui a changé avant de parler du classement.

V- Afficher les cinq produits les plus chers

8. Inversez le sens du tri sur listprice en conservant le départage par productid.

```
SELECT productid, name, listprice
FROM production.product
WHERE listprice IS NOT NULL
ORDER BY listprice DESC, productid ASC
LIMIT 5;
```

9. Contrôlez que les prix ne remontent jamais lorsque vous descendez dans la grille. En cas d'égalité, vérifiez la progression de productid.

LIMIT 5 sélectionne au maximum cinq lignes, pas cinq prix différents. Plusieurs variantes d'un produit peuvent partager un prix. Si de nombreux produits ont le même prix à la frontière, LIMIT n'ajoute pas automatiquement tous les ex æquo. Ce TP classe les lignes ; le classement par rang sera étudié ultérieurement.

VI- Définir la place des valeurs NULL

Dans PostgreSQL, les valeurs NULL sont placées par défaut à la fin d'un tri ASC et au début d'un tri DESC. NULLS FIRST ou NULLS LAST permet de choisir explicitement leur position. NULL n'est ni zéro ni une chaîne vide.

10. Triez les couleurs en plaçant les valeurs absentes à la fin.

```
SELECT productid, name, color
FROM production.product
ORDER BY color ASC NULLS LAST, productid ASC;
```

11. Remplacez NULLS LAST par NULLS FIRST puis comparez les premières lignes. Vous pouvez ajouter LIMIT 10 pour faciliter la lecture.

Pour des prix décroissants, ORDER BY listprice DESC NULLS LAST place les prix inconnus après les prix connus. Si la consigne porte sur les produits dont le prix est connu, WHERE listprice IS NOT NULL reste la manière explicite de les sélectionner.

VII- Trier sur plusieurs colonnes

Le premier critère est prioritaire. Le deuxième ne départage que les lignes égales sur le premier ; le troisième ne départage que celles égales sur les deux premiers. Les colonnes sont séparées par des virgules. Ce tri ne constitue pas une agrégation : aucune ligne n'est fusionnée.

12. Triez les sous-catégories en ordre croissant et les prix en ordre décroissant à l'intérieur de chacune.

```
SELECT productid, name, productsubcategoryid, listprice
FROM production.product
ORDER BY productsubcategoryid ASC NULLS LAST,
         listprice DESC NULLS LAST, productid ASC;
```

13. Repérez deux lignes de la même sous-catégorie, puis le passage à la suivante. Vérifiez que le prix peut remonter au changement de sous-catégorie : la priorité appartient au premier critère.

14. Ajoutez LIMIT 100. Vérifiez que vous obtenez au maximum les cent premières lignes de la liste globale triée.

Cette requête ne sélectionne pas cent produits par sous-catégorie. LIMIT s'applique au résultat entier. La sélection des meilleurs produits dans chaque groupe nécessite d'autres techniques, hors du périmètre de cette séance.

15. Observez un exemple de tri mixte indépendant des données AdventureWorks.

```
SELECT prenom, nom
FROM (VALUES ('John', 'Doe'), ('Jane', 'Smith'),
             ('John', 'Smith'), ('Jane', 'Doe'),
             ('John', 'Johnson'), ('Jane', 'Johnson'))
     AS personnes(prenom, nom)
ORDER BY prenom ASC, nom DESC;
```

Dans l'ordre alphabétique usuel de ces noms, Jane précède John. Pour chaque prénom, Smith précède Johnson, puis Doe, car nom est décroissant. VALUES construit ce petit jeu dans la requête sans créer de table ni modifier les données.

VIII- Comprendre LIMIT et ses limites

16. Reprenez une requête triée et remplacez successivement LIMIT 10 par LIMIT 3 puis LIMIT 0.

LIMIT 3 restitue au maximum trois lignes ; LIMIT 0 ne restitue aucune ligne. Si moins de lignes satisfont les filtres, le résultat en contient moins que la limite demandée. LIMIT sans ORDER BY est autorisé, mais ne définit pas quelles lignes doivent être les premières.

PostgreSQL n'impose pas une limite universelle de mille lignes à toutes les requêtes SELECT. La grille pgAdmin peut charger des résultats progressivement ou être configurée avec des limites propres. LIMIT est une restriction explicite dans le SQL. Il réduit les lignes transmises ; le travail du serveur dépend aussi du filtre, du tri et des index, et peut encore nécessiter la lecture de nombreuses lignes.

IX- Parcourir une liste avec OFFSET

OFFSET indique combien de lignes triées sont sautées avant de restituer la page. Pour LIMIT n OFFSET k, les positions affichées vont de k + 1 à k + n, si assez de lignes existent. Ces positions ne sont pas des identifiants de produits.

17. Affichez d'abord les dix premières lignes des prix connus en ordre décroissant.

```
SELECT productid, name, listprice
FROM production.product
WHERE listprice IS NOT NULL
ORDER BY listprice DESC, productid ASC
LIMIT 10;
```

18. Affichez uniquement les positions 6 à 10 de la même liste.

```
SELECT productid, name, listprice
FROM production.product
WHERE listprice IS NOT NULL
ORDER BY listprice DESC, productid ASC
LIMIT 5 OFFSET 5;
```

Les cinq premières lignes sont ignorées, puis cinq lignes au maximum sont retournées. Comparez productid avec les positions 6 à 10 de la requête précédente. Conservez exactement les mêmes filtres et critères de tri pour comparer les pages.

19. Affichez les positions 11 à 20 avec LIMIT 10 OFFSET 10, puis les positions 21 à 30 avec LIMIT 10 OFFSET 20.

X- Synthèse et contrôles

Avant d'exécuter, vérifiez le périmètre du WHERE, la priorité des critères, le sens propre à chaque colonne, la place des NULL, le départage des ex æquo et la taille de page. Contrôlez les identifiants autant que les valeurs de prix. Sans accès à votre serveur, les exemples de ce support restent des requêtes à exécuter et ne prétendent pas reproduire les effectifs de votre sauvegarde.

Travail pratique en classe

1. Sélectionnez les 10 premiers employés embauchés en utilisant la table 'humanresources.employee'. – 10 rows

2. Sélectionnez les 5 derniers employés embauchés en utilisant la table 'humanresources.employee'. – 5 rows

3. Sélectionnez les 100 employés, classés par jobtitle, puis par date d'arrivée, avec la table 'humanresources.employee'. – 100 rows

4. Sélectionnez les 10 employés, classés par jobtitle, puis par date d'arrivée, avec la table 'humanresources.employee', et isoler les lignes 10 à 20. – 10 rows

Travail pratique à rendre

Exercice 1- Comprendre le classement

1. Expliquez ASC, DESC et le sens par défaut.

2. Pourquoi LIMIT 5 sans ORDER BY ne définit-il pas les cinq produits les moins chers ?

3. Pourquoi ajouter productid après un tri par prix ?

4. Donnez les positions restituées par LIMIT 5 OFFSET 5 et LIMIT 10 OFFSET 20.

Exercice 2- Trier le catalogue

Affichez les colonnes utiles au contrôle et départagez les ex æquo par productid.

Q1. Les dix premiers noms par ordre alphabétique croissant. – 10 rows

Q2. Les cinq produits les moins chers à prix strictement positif. – 5 rows

Q3. Les cinq produits les plus chers à prix connu. – 5 rows

Q4. Tous les produits par sous-catégorie croissante, puis prix décroissant, valeurs NULL à la fin. – 504 rows

Q5. Les vingt premiers produits Black de prix supérieur à 100, par prix croissant. – 20 rows

Exercice 3- Paginer et comparer

Q6. Afficher les positions 6 à 10 dans la liste des produits à prix connu, par prix décroissant et identifiant croissant. – 5 rows

Q7. Afficher les positions 21 à 30 de cette même liste. – 10 rows

Q8. Afficher la page 4 de sept lignes et justifier le décalage. – 7 rows

Q9. Trier les couleurs par ordre décroissant avec les valeurs absentes à la fin, puis par nom et identifiant croissants ; limiter à quinze lignes. – 15 rows

Q10. Comparer les deux premières pages de cinq produits avec le même tri, et expliquer le contrôle des identifiants pour des données inchangées. – 5, 5 rows

Exercice 4- Diagnostiquer et corriger

Expliquez chaque défaut, puis corrigez selon la consigne.

```
-- A : les cinq produits les plus chers à prix connu
SELECT productid, name, listprice FROM production.product
LIMIT 5 ORDER BY listprice DESC;

-- B : prix decroissant dans chaque sous-categorie
SELECT productid, productsubcategoryid, listprice
FROM production.product
ORDER BY listprice DESC, productsubcategoryid ASC;

-- C : positions 6 a 10 par prix decroissant
SELECT productid, name, listprice FROM production.product
ORDER BY listprice DESC LIMIT 10 OFFSET 5;

-- D : pagination stable en cas de prix egaux
SELECT productid, name, listprice FROM production.product
ORDER BY listprice DESC LIMIT 5 OFFSET 5;
```

Pour A, C et D, précisez aussi le traitement des prix NULL. Dans D, une syntaxe valide ne garantit pas le choix reproductible des lignes lorsque plusieurs produits ont le même prix.

Exercice 5- Présenter les contrôles

Remettez TP8_Nom_Prenom.sql avec Q1 à Q10 et les corrections A à D, ainsi qu'un court rapport TP8_Nom_Prenom.pdf. Incluez votre identité, les réponses théoriques, deux captures du code et des résultats de Q4 et Q6 et la vérification des pages Q10. Commentez les requêtes et indiquez les éventuelles adaptations.